

$$f(q, m) \cdot f(k, n) = g(q, k, n - m)$$

位置编码极简书

从正弦编码到 RoPE、YaRN 与 PaTH
37 种位置编码方法的原理、公式与可运行代码

作者：刘玉书

37 节 · 公式 · PyTorch

目录

第 0 章 约定与公共代码 **1**

第 1 章 绝对位置编码 **3**

1.1 正弦位置编码（Sinusoidal） . . . 3
1.2 可学习绝对位置嵌入（Learned Absolute） . . . 3
1.3 随机位置编码（Randomized Positional Encodings） . . . 4

第 2 章 相对位置偏置 **5**

2.1 Shaw 相对位置表示 . . . 5
2.2 Transformer-XL 相对位置 . . . 5
2.3 T5 相对位置偏置 . . . 6
2.4 DeBERTa 解耦注意力（Disentangled Attention） . . . 6
2.5 ALiBi（线性偏置注意力） . . . 7
2.6 KERPLE（核化相对位置） . . . 7
2.7 Sandwich . . . 7
2.8 FIRE（函数插值相对位置） . . . 8

第 3 章 旋转位置编码 RoPE 及其变体 **9**

3.1 旋转位置编码（RoPE） . . . 9
3.2 xPos（可外推的 RoPE） . . . 9
3.3 部分 RoPE（Partial RoPE） . . . 10
3.4 解耦 RoPE（Decoupled RoPE, MLA） . . . 10
3.5 二维 RoPE（2D RoPE，视觉） . . . 10
3.6 多模态 RoPE（M-RoPE） . . . 11
3.7 交错式 M-RoPE（Interleaved MRoPE） . . . 11
3.8 VideoRoPE . . . 12

第 4 章 RoPE 长度扩展 **13**

4.1 调大 base（ABF, Adjusted Base Frequency） . . . 13
4.2 位置插值（Position Interpolation, PI） . . . 13
4.3 NTK-aware 插值 . . . 14
4.4 动态 NTK（Dynamic NTK） . . . 14
4.5 YaRN . . . 14
4.6 LongRoPE / LongRoPE2 . . . 15
4.7 Self-Extend . . . 15

4.8 ReRoPE / Leaky ReRoPE 16
4.9 双块注意力（Dual Chunk Attention, DCA） 16
4.10 LogN 注意力缩放 17

第 5 章 无位置编码与混合方案 **18**
5.1 无位置编码（NoPE） 18
5.2 NoPE / RoPE 交错（iRoPE、RNoPE） 18
5.3 局部层与全局层使用不同 base 19

第 6 章 数据依赖的位置编码 **20**
6.1 上下文位置编码（Contextual Position Encoding, CoPE） 20
6.2 PaTH 注意力 20
6.3 遗忘门作为位置（Forgetting Transformer, FoX） 21
6.4 Stick-Breaking 注意力 21

第 7 章 隐式位置 **23**
7.1 递推结构自带位置（RNN / SSM / 线性注意力） 23

附录 速查表 **24**

第 0 章 约定与公共代码

为什么需要位置编码：自注意力对输入是**置换等变**的。把输入的 token 打乱顺序，输出只是跟着打乱，模型本身分不出“猫追狗”和“狗追猫”。位置编码就是把顺序信息注入模型的手段。

记号：B 批大小，H 头数，N 序列长度，d 每头维度，$D = Hd$ 模型维度；i（或 m）为查询位置，j（或 n）为键位置；L 为训练长度，L' 为目标长度，$s = L'/L$ 为扩展倍数。

位置信息可以注入的四个地方：

$$\text{Attn} = \text{softmax}\left(\frac{\overbrace{(x_i + p_i)}^{\text{① 输入端}} W_Q \quad \overbrace{\widetilde{R}_\Theta}^{\text{③ 旋转 q/k}} \quad \left((x_j + p_j) W_K\right)^\top}{\sqrt{d}} + \underbrace{b(i,j)}_{\text{② 打分偏置}} \right) V$$

④ 什么都不加，靠因果掩码或递推结构隐式获得

公共代码：后文每个代码块都默认已运行下面这段（`pip install torch` 即可，CPU 上可跑）。

```
import math, torch, torch.nn as nn, torch.nn.functional as F
torch.manual_seed(0)
B, H, N, d = 2, 4, 16, 32                    # 批、头数、序列长度、每头维度
D = H * d                                    # 模型维度
x = torch.randn(B, N, D)                     # 输入序列 (B,N,D)
q, k, v = (torch.randn(B, H, N, d) for _ in range(3))   # 已分好头的 Q/K/V
pos = torch.arange(N)                        # 位置下标 0..N-1
I, J = pos[:, None], pos[None]               # 查询位置 i、键位置 j
causal = I >= J                              # 因果掩码，True = 可见

def sdpa(q, k, v, mask=None, bias=None):
    """缩放点积注意力。mask 中 True 表示可见；bias 加到打分上。"""
    s = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
    if bias is not None: s = s + bias
    if mask is not None: s = s.masked_fill(~mask, float('-inf'))
    return s.softmax(-1) @ v

def rope_freqs(dim, base=10000.0):
    """RoPE 的频率 θ_i = base^(-2i/dim)，共 dim/2 个。"""
    return base ** (-torch.arange(0, dim, 2).float() / dim)

def apply_rot(t, ang):
    """把 t 的相邻两维 (2i, 2i+1) 看成一个复数，旋转角度 ang（形状可广播到 (...,N,dim/2)）。"""
    t1, t2 = t[..., ::2], t[..., 1::2]
```

```
    c, s = ang.cos(), ang.sin()
    return torch.stack([t1 * c - t2 * s, t1 * s + t2 * c], -1).flatten(-2)

def rope(t, positions=None, base=10000.0):
    """标准 RoPE：第 p 个位置旋转 p·θ_i。positions 可传入任意（含小数）位置。"""
    p = torch.arange(t.size(-2)) if positions is None else positions
    return apply_rot(t, p.float()[..., None] * rope_freqs(t.size(-1), base))
```

示例代码只为说明原理：用最直白的写法，省略了缓存、融合 kernel 和数值细节。工程实现请参考 Hugging Face `transformers` 中的 `rope_utils`、vLLM 等。

第 1 章 绝对位置编码

给每个位置 p 一个向量 p_p，在进入第一层之前加到 token 嵌入上。这种做法简单直观，但模型看到的是“第几个”，而不是“相距多远”，外推到训练长度以外通常很差。

1.1 正弦位置编码（Sinusoidal）

Vaswani et al., 2017 · 原始 Transformer

不同维度对应不同频率的正余弦（波长从 2π 到 $10000 \cdot 2\pi$ 呈几何级数），无需训练。它有一个性质：PE_{p+k} 是 PE_p 的线性函数（一个只依赖 k 的旋转），理论上模型可以从中学到相对位置。

$$PE_{(p,\,2i)} = \sin\!\left(\frac{p}{10000^{2i/D}}\right), \qquad PE_{(p,\,2i+1)} = \cos\!\left(\frac{p}{10000^{2i/D}}\right)$$

```
def sinusoidal(n, dim):
    p = torch.arange(n).float()[:, None]
    inv = 10000 ** (-torch.arange(0, dim, 2).float() / dim)
    pe = torch.zeros(n, dim)
    pe[:, 0::2], pe[:, 1::2] = torch.sin(p * inv), torch.cos(p * inv)
    return pe
pe = sinusoidal(N, D)
out = x + pe                                              # 加到输入嵌入上
# 性质: PE[p+k] 是 PE[p] 的旋转，旋转矩阵只依赖 k
kk, inv = 3, 10000 ** (-torch.arange(0, D, 2).float() / D)
c, s = torch.cos(kk * inv), torch.sin(kk * inv)
sin_p, cos_p = pe[:-kk, 0::2], pe[:-kk, 1::2]
print(torch.allclose(pe[kk:, 0::2], sin_p * c + cos_p * s, atol=1e-5))
```

1.2 可学习绝对位置嵌入（Learned Absolute）

Devlin et al., 2019（BERT）；Radford et al., 2019（GPT-2）· GPT-3、OPT 使用

每个位置一个可训练向量，存成一张大小为 $L_{\max} \times D$ 的表。表达能力强，但**完全无法处理超过 $L_{\max}$ 的位置**。

$$h_p^{(0)} = x_p + P_p, \qquad P \in \mathbb{R}^{L_{\max} \times D}$$

```
L_max = 512
P = nn.Embedding(L_max, D)                              # 可学习位置表
out = x + P(pos)
try:
    P(torch.tensor([L_max]))                            # 超出训练长度
except IndexError as e:
    print(out.shape, "| 位置 512 超出表长: ", type(e).__name__)
```

1.3 随机位置编码（Randomized Positional Encodings）

Ruoss et al., 2023 · ACL

为了让模型在测试时见过“大位置”，训练时不使用 $0, 1, \dots, N-1$，而是从更大的范围 $[0, L_{\max})$ 里随机抽取 N 个位置并排序。这样模型只学到位置之间的**先后关系**，长度泛化能力显著改善。

$$(p_1 < p_2 < \cdots < p_N) \sim \mathrm{SortedSample}(\{0, \dots, L_{\max}-1\}, N), \qquad h_t^{(0)} = x_t + PE_{p_t}$$

```
def sinusoidal(n, dim):
    p = torch.arange(n).float()[:, None]
    inv = 10000 ** (-torch.arange(0, dim, 2).float() / dim)
    return torch.cat([torch.sin(p * inv), torch.cos(p * inv)], -1)
L_max = 256
table = sinusoidal(L_max, D)
rand_pos = torch.randperm(L_max)[:N].sort().values      # 训练时：随机抽取并排序
out = x + table[rand_pos]                               # 测试时：直接用 0..N'-1（N' 可远大于 N）
print(rand_pos.tolist())
```

第 2 章 相对位置偏置

注意力只关心“两个 token 相距多远”，于是直接把相对距离 $i-j$ 的函数加到注意力打分上。从 Shaw 的向量形式，到 T5、ALiBi 的标量偏置，再到 KERPLE、FIRE 这类可学习的函数形式，长度外推能力一路提升。

2.1 Shaw 相对位置表示

Shaw et al., 2018

最早的相对位置方法：为每个（截断后的）相对距离学一个向量，加到 key 上参与点积，也可以同时加到 value 上。

$$e_{ij}=\frac{q_i^\top\left(k_j+a^K_{\mathrm{clip}(j-i,-c,c)}\right)}{\sqrt{d}},\qquad o_i=\sum_j\alpha_{ij}\left(v_j+a^V_{\mathrm{clip}(j-i,-c,c)}\right)$$

```
c = 4
rel = (J - I).clamp(-c, c) + c                          # 相对距离 → 下标 [0, 2c]
aK, aV = nn.Embedding(2 * c + 1, d)(rel), nn.Embedding(2 * c + 1, d)(rel)   # (N,N,d)
s = (q @ k.transpose(-2, -1) + torch.einsum('bhid,ijd->bhij', q, aK)) / math.sqrt(d)
A = s.softmax(-1)
out = A @ v + torch.einsum('bhij,ijd->bhid', A, aV); print(out.shape)
```

2.2 Transformer-XL 相对位置

Dai et al., 2019

把“绝对位置编码代入点积后展开的 4 项”中的绝对位置全部换成相对距离 R_{i-j}（正弦向量），并把查询侧的位置项换成两个全局可学习向量 u、v。XLNet 也采用了这种形式。

$$s_{ij}=\underbrace{q_i^\top k_j}_{\text{内容-内容}}+\underbrace{q_i^\top W_R R_{i-j}}_{\text{内容-位置}}+\underbrace{u^\top k_j}_{\text{全局内容偏置}}+\underbrace{v^\top W_R R_{i-j}}_{\text{全局位置偏置}}$$

```
def sinus(p, dim):                                      # 对任意（可为负）距离求正弦向量
    ang = p.float()[..., None] * 10000 ** (-torch.arange(0, dim, 2).float() / dim)
    return torch.cat([ang.sin(), ang.cos()], -1)
W_R = nn.Linear(d, d, bias=False)
R = W_R(sinus(I - J, d))                                # (N,N,d)
```

```
u, vb = torch.randn(H, 1, d), torch.randn(H, d)           # 全局可学习向量
s = (q @ k.transpose(-2, -1)                              # 内容-内容
     + torch.einsum('bhid,ijd->bhij', q, R)               # 内容-位置
     + u @ k.transpose(-2, -1)                            # 全局内容偏置
     + torch.einsum('hd,ijd->hij', vb, R)[None]) / math.sqrt(d)   # 全局位置偏置
out = s.masked_fill(~causal, float('-inf')).softmax(-1) @ v; print(out.shape)
```

2.3 T5 相对位置偏置

Raffel et al., 2020 · T5、Flan-T5

每个头为每个“相对距离桶”学一个**标量**，直接加到打分上。距离 < 8 时一个距离一个桶，再远则按对数分桶，超过 max_distance 后全部归入最后一个桶。偏置只在第一层计算，之后各层共享。

$$s_{ij} = q_i^\top k_j + b_h\big(\mathrm{bucket}(j-i)\big), \qquad \mathrm{bucket}(n) = \begin{cases} n & n < \frac{n_b}{2} \\ \frac{n_b}{2} + \left\lfloor \frac{\log(n/\frac{n_b}{2})}{\log(d_{\max}/\frac{n_b}{2})} \cdot \frac{n_b}{2} \right\rfloor & \text{否则} \end{cases}$$

```
def t5_bucket(rel, num_buckets=32, max_distance=128):     # 双向版本，与官方实现一致
    nb, n = num_buckets // 2, -rel
    ret = (n < 0).long() * nb                             # 前后两个方向各一半桶
    n = n.abs()
    max_exact = nb // 2
    large = max_exact + (torch.log(n.float() / max_exact + 1e-6)
                         / math.log(max_distance / max_exact) * (nb - max_exact)).long()
    return ret + torch.where(n < max_exact, n, large.clamp(max=nb - 1))
bucket = t5_bucket(J - I)
bias = nn.Embedding(32, H)(bucket).permute(2, 0, 1)       # (H,N,N)
out = sdpa(q, k, v, bias=bias)
print(t5_bucket(torch.tensor([0, -1, -7, -8, -20, -200, 5])).tolist())
```

2.4 DeBERTa 解耦注意力（Disentangled Attention）

He et al., 2021 · DeBERTa

内容和相对位置用**两套独立的向量**表示，注意力打分由三项构成：内容-内容、内容→位置、位置→内容（省略位置-位置项）。缩放因子相应改为 $\sqrt{3d}$。

$$\tilde{A}_{ij} = \underbrace{Q_i^c K_j^{c\top}}_{\text{内容-内容}} + \underbrace{Q_i^c K_{\delta(i,j)}^{r\top}}_{\text{内容→位置}} + \underbrace{K_j^c Q_{\delta(j,i)}^{r\top}}_{\text{位置→内容}}, \qquad \delta(i,j) = \mathrm{clip}(i-j, -\kappa, \kappa-1) + \kappa$$

```
kap = 4
Prel = nn.Embedding(2 * kap, d).weight                    # 共享的相对位置嵌入
Kr, Qr = nn.Linear(d, d)(Prel), nn.Linear(d, d)(Prel)     # 位置的 key / query 投影
delta = (I - J).clamp(-kap, kap - 1) + kap                # δ(i,j)
c2c = q @ k.transpose(-2, -1)
c2p = torch.einsum('bhid,ijd->bhij', q, Kr[delta])        # Q^c_i · K^r_{δ(i,j)}
p2c = torch.einsum('bhjd,jid->bhij', k, Qr[delta])        # K^c_j · Q^r_{δ(j,i)}
out = ((c2c + c2p + p2c) / math.sqrt(3 * d)).softmax(-1) @ v; print(out.shape)
```

2.5 ALiBi（线性偏置注意力）

Press et al., 2022 · ICLR · BLOOM、MPT、Baichuan2-13B 使用

完全不加位置向量，只在打分上减去一个**与距离成正比**的惩罚，每个头的斜率 m_h 构成几何级数。这样做实现极简，“训短测长”的外推能力强。

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} - m_h \cdot (i-j), \qquad m_h = 2^{-8h/H},\ h = 1, \dots, H$$

```
m = 2.0 ** (-8 * torch.arange(1, H + 1) / H)              # H=4 → 1/4, 1/16, 1/64, 1/256
bias = -m[:, None, None] * (I - J).clamp(min=0)           # (H,N,N)
out = sdpa(q, k, v, causal, bias)
print(m.tolist())
```

2.6 KERPLE（核化相对位置）

Chi et al., 2022 · NeurIPS

把 ALiBi 推广为**条件正定核**：偏置是可学习参数 $r_1, r_2 > 0$ 下距离的函数。对数形式在长度外推上效果最好，ALiBi 是幂形式在 $r_2 = 1$ 时的特例。

$$\text{log 变体: } b_{ij} = -r_1 \log\left(1 + r_2 \left|i-j\right|\right) \qquad \text{power 变体: } b_{ij} = -r_1 \left|i-j\right|^{r_2}$$

```
r1 = F.softplus(torch.randn(H, 1, 1)) + 0.1               # 每头可学习，保持为正
r2 = F.softplus(torch.randn(H, 1, 1)) + 0.1
dist = (I - J).clamp(min=0).float()
bias_log = -r1 * torch.log1p(r2 * dist)                   # 对数变体
bias_pow = -r1 * dist ** r2.clamp(max=2)                  # 幂变体（r2∈(0,2]）
out = sdpa(q, k, v, causal, bias_log); print(out.shape)
```

2.7 Sandwich

Chi et al., 2023 · ACL

没有任何可学习参数：直接用正弦位置向量的**内积** $p_i^\top p_j$ 作为偏置。它只依赖 $i-j$，并且随距离近似单调衰减（类似 ALiBi 的感受野收缩）。作者用这个方法解释了为什么 ALiBi、KERPLE 能够外推。

$$b_{ij} = \lambda_h \, p_i^\top p_j = \lambda_h \sum_{k=1}^{d'/2} \cos\left(\frac{i-j}{10000^{2k/d'}}\right)$$

```
dp = 64
inv = 10000 ** (-torch.arange(1, dp // 2 + 1).float() * 2 / dp)
bias = torch.cos((I - J).float()[..., None] * inv).sum(-1)              # = p_i · p_j
bias = (bias - dp / 2) * (8.0 / H) * torch.arange(1, H + 1)[:, None, None] / H   # 减去 d'/2 使 b_ii=0，按头缩放
out = sdpa(q, k, v, causal, bias)
print([round(x, 2) for x in bias[0, -1, -6:].tolist()])                 # 距离越远，值越负
```

2.8 FIRE（函数插值相对位置）

Li et al., 2024 · ICLR · Google

用一个小 MLP f_θ 把相对距离映射为偏置，但先做**渐进插值**：把距离除以查询位置（取对数变换后），使输入始终落在 $[0, 1]$ 内。这样无论序列多长，MLP 都不会遇到训练时没见过的输入。T5 偏置、ALiBi、KERPLE 都可以看作 FIRE 的特例。

$$b(i,j) = f_\theta\left(\frac{\psi(i-j)}{\psi(\max(L_{\text{th}},\, i))}\right), \qquad \psi(x) = \log(cx+1)$$

```
c, L_th = 1.0, 8.0                                      # c、L_th 原文均可学习
psi = lambda t: torch.log(c * t + 1)
f = nn.Sequential(nn.Linear(1, 32), nn.ReLU(), nn.Linear(32, H))   # 每头输出一个偏置
rel = (I - J).clamp(min=0).float()
norm = psi(rel) / psi(torch.clamp(I.float(), min=L_th))    # 归一化到 [0,1]，与绝对长度无关
bias = f(norm[..., None]).permute(2, 0, 1)                 # (H,N,N)
out = sdpa(q, k, v, causal, bias)
print(f"输入范围: [{norm[causal].min():.2f}, {norm[causal].max():.2f}]")
```

第 3 章 旋转位置编码 RoPE 及其变体

RoPE 用**绝对位置**去旋转 q 和 k，做点积后却只留下**相对位置**，同时兼得两类方法的优点，因此成为当今 LLM 的事实标准。本章介绍 RoPE 本身，以及它在外推、KV 压缩、多模态等方向上的变体。

3.1 旋转位置编码（RoPE）

Su et al., 2021（RoFormer）· Llama、Qwen、Mistral、DeepSeek、Gemma 等几乎所有开源 LLM 使用

把 q、k 的每两维看成一个复数，第 m 个位置旋转 $m\theta_i$ 角度。由于旋转矩阵满足 $R_m^\top R_n = R_{n-m}$，内积只依赖相对位置 $n-m$。高频维度（i 小）对近距离敏感，低频维度负责远距离；并且注意力天然带有随距离衰减的趋势。

$$f(x,m)_{[2i:2i+2]} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} x_{[2i:2i+2]}, \qquad \theta_i = \text{base}^{-2i/d}$$

$$\langle f(q,m), f(k,n)\rangle = g(q,k,n-m)$$

```
qr, kr = rope(q), rope(k)                                   # 公共代码中的实现
out = sdpa(qr, kr, v, causal)
# 等价的复数写法：x · e^{i m θ}
qc = torch.view_as_complex(q.reshape(*q.shape[:-1], -1, 2))
qr2 = torch.view_as_real(qc * torch.polar(torch.ones(N, d // 2), pos[:, None] * rope_freqs(d))).flatten(-2)
print("复数写法一致:", torch.allclose(qr, qr2, atol=1e-5))
# 相对性：把同一对 (q,k) 放在 (m,n)=(5,2) 与 (12,9)，内积相同
q0, k0 = q[0, 0, :1], k[0, 0, :1]
f = lambda t, p: rope(t, torch.tensor([p]))
print("只依赖 n-m:", torch.allclose(f(q0, 5) @ f(k0, 2).T, f(q0, 12) @ f(k0, 9).T, atol=1e-4))
```

3.2 xPos（可外推的 RoPE）

Sun et al., 2023 · ACL · “A Length-Extrapolatable Transformer”

RoPE 的注意力在长距离上会剧烈振荡。xPos 在旋转之外再给 q 乘上 $\zeta^{m/\gamma}$、给 k 乘上 $\zeta^{-n/\gamma}$：两者相乘后得到只依赖 $m-n$ 的指数衰减。由于 $\zeta_i < 1$，越远的 token 贡献越小，振荡被抑制。

$$\tilde{q}_m = \zeta^{m/\gamma} \odot f(q,m), \quad \tilde{k}_n = \zeta^{-n/\gamma} \odot f(k,n), \quad \tilde{q}_m^\top \tilde{k}_n \propto \zeta^{(m-n)/\gamma}, \qquad \zeta_i = \frac{2i/d + 0.4}{1.4}$$

```
gamma = 512                                                 # scale_base
zeta = (torch.arange(0, d, 2).float() / d + 0.4) / 1.4      # 每个频率对一个衰减率 ∈ (0.29, 1)
sc = (zeta ** (pos.float()[:, None] / gamma)).repeat_interleave(2, -1)   # (N,d)，对应交错的维度对
qx, kx = rope(q) * sc, rope(k) / sc                         # q 乘、k 除 → 乘积只含 ζ^{(m-n)/γ}
out = sdpa(qx, kx, v, causal); print(out.shape)
```

3.3 部分 RoPE（Partial RoPE）

Black et al., 2022（GPT-NeoX，旋转 25% 维度）；GPT-J（rotary_dim=64）；Phi-2 等

只对每个头的前 $r \cdot d$ 维做旋转，其余维度保持不变。不旋转的维度可以做纯语义匹配，不受距离衰减影响，同时也省一点计算。

$$f_{\text{partial}}(x, m) = \left[\, f(x_{[:rd]}, m)\,;\; x_{[rd:]} \,\right]$$

```
rot = int(0.25 * d)                                         # 只旋转 25% 的维度
prope = lambda t: torch.cat([rope(t[..., :rot]), t[..., rot:]], -1)
out = sdpa(prope(q), prope(k), v, causal)
print("不旋转部分保持不变:", torch.equal(prope(q)[..., rot:], q[..., rot:]))
```

3.4 解耦 RoPE（Decoupled RoPE, MLA）

DeepSeek-V2, 2024 · DeepSeek-V3/R1、Kimi K2 使用

MLA 把 K/V 压缩成潜向量 c_t，推理时希望把上投影 W^{UK} “吸收”进查询，直接拿 q 与 c 做点积。但 RoPE 夹在两者之间，矩阵乘法无法交换，吸收就失效了。解决办法是把位置信息拆到一条**单独的小分支**：内容部分不加位置编码，可以吸收；位置部分维度很小（d_r），所有头共享同一个 RoPE key。

$$q_i = [\, q_i^C;\; f(q_i^R, i)\,], \qquad k_j = [\, W^{UK} c_j;\; f(W^{KR} h_j, j)\,]$$

$$q_i^\top k_j = \underbrace{(W^{UK\top} q_i^C)^\top c_j}_{\text{可吸收}} + f(q_i^R, i)^\top f(k_j^R, j)$$

```
dc, dr = 16, 8
c_lat = torch.randn(B, N, dc)                               # KV 潜向量（cache）
W_uk = torch.randn(H, d, dc) / dc ** .5                     # 每头的上投影
qC, qR = torch.randn(B, H, N, d), torch.randn(B, H, N, dr)
kR = torch.randn(B, 1, N, dr)                               # RoPE key：所有头共享，只缓存一份
kC = torch.einsum('hdc,bnc->bhnd', W_uk, c_lat)             # 显式还原 K
s_explicit = qC @ kC.transpose(-2, -1)
s_absorb = torch.einsum('bhid,hdc,bjc->bhij', qC, W_uk, c_lat)    # 吸收：q 先乘 W_uk^T，再与 c 点积
s = s_absorb + rope(qR) @ rope(kR).transpose(-2, -1)
out = (s / math.sqrt(d + dr)).masked_fill(~causal, float('-inf')).softmax(-1) @ v
print("吸收后内容分数一致:", torch.allclose(s_explicit, s_absorb, atol=1e-4))
```

3.5 二维 RoPE（2D RoPE，视觉）

Heo et al., 2024（RoPE-ViT）；Qwen2-VL 视觉编码器等

图像块有行、列两个坐标。把每头的维度一分为二：前一半按行号 y 旋转，后一半按列号 x 旋转，点积就只依赖二维相对位移 $(\Delta y, \Delta x)$。RoPE-Mixed 还进一步让每个频率同时混合 x 和 y 两个方向。

$$f_{2D}(t, y, x) = \left[\, f(t_{[:d/2]},\, y)\;;\; f(t_{[d/2:]},\, x) \,\right]$$

```
Hp = Wp = 4                                                 # 4×4 个图像块 = N
ys, xs = pos // Wp, pos % Wp                                # 每个 token 的 (行, 列)
rope2d = lambda t: torch.cat([rope(t[..., :d // 2], ys), rope(t[..., d // 2:], xs)], -1)
out = sdpa(rope2d(q), rope2d(k), v)
# 只依赖二维位移：(0,0)-(1,2) 与 (2,1)-(3,3) 的位移都是 (1,2)
q0, k0 = q[0, 0, :1], k[0, 0, :1]
g = lambda t, y, x: torch.cat([rope(t[..., :d // 2], torch.tensor([y])), rope(t[..., d // 2:],
↪  torch.tensor([x]))], -1)
print(torch.allclose(g(q0, 1, 2) @ g(k0, 0, 0).T, g(q0, 3, 3) @ g(k0, 2, 1).T, atol=1e-4))
```

3.6 多模态 RoPE（M-RoPE）

Wang et al., 2024 · Qwen2-VL；Qwen2.5-VL（时间维对齐绝对时间）

把位置拆成**时间** t、**高度** h、**宽度** w 三个分量，频率维度按区段分给三者（Qwen2-VL 中 128 维头的分配为 16/24/24 个频率对）。三类 token 的处理方式：

- **文本 token**：三个分量取相同值，退化为普通 1D RoPE
- **图像 token**：t 固定，h、w 取网格坐标
- **图像之后的文本**：从前面最大位置 $+1$ 继续编号

$$\phi_i(t,h,w) = \begin{cases} t\,\theta_i & i \in \mathcal{S}_t \\ h\,\theta_i & i \in \mathcal{S}_h \\ w\,\theta_i & i \in \mathcal{S}_w \end{cases}, \qquad f(x;t,h,w)_{[2i:2i+2]} = R(\phi_i)\, x_{[2i:2i+2]}$$

```
# 序列：4 个文本 token + 3×3 图像 + 3 个文本 token = 16
t_ids, h_ids, w_ids = [], [], []
for p in range(4): t_ids.append(p); h_ids.append(p); w_ids.append(p)        # 文本：三者相同
for r in range(3):
    for c in range(3): t_ids.append(4); h_ids.append(4 + r); w_ids.append(4 + c)   # 图像：t 固定
for p in range(7, 10): t_ids.append(p); h_ids.append(p); w_ids.append(p)    # 之后的文本从 max+1 继续
ids = torch.tensor([t_ids, h_ids, w_ids]).float()          # (3,N)
sec = [4, 6, 6]                                            # d/2=16 个频率对的区段划分（t/h/w）
axis = torch.repeat_interleave(torch.arange(3), torch.tensor(sec))   # 每个频率对用哪个坐标
ang = ids[axis].T * rope_freqs(d)                          # (N, d/2)
mrope = lambda t: apply_rot(t, ang)
out = sdpa(mrope(q), mrope(k), v, causal)
print(ids[:, 4:13].long().tolist())
```

3.7 交错式 M-RoPE（Interleaved MRoPE）

Qwen3-VL, 2025

M-RoPE 按**区段**分配频率，导致时间维只拿到最高频的一段，而高度、宽度只拿到中低频段，频谱分配不均，长视频理解受影响。交错式改为按 $t, h, w, t, h, w, \ldots$ **轮流**分配频率，三个坐标轴都覆盖从高频到低频

的完整频谱。

$$\text{axis}(i) = (t, h, w)_{\,i \bmod 3}, \qquad \phi_i = \text{pos}_{\text{axis}(i)} \cdot \theta_i$$

```
ids = torch.stack([torch.full((N,), 4.), (pos // 4).float() + 4, (pos % 4).float() + 4])   # 一张 4×4 图像
axis_chunk = torch.repeat_interleave(torch.arange(3), torch.tensor([6, 5, 5]))   # 区段式（M-RoPE）
axis_inter = torch.arange(d // 2) % 3                                             # 交错式
for name, ax in [("区段式", axis_chunk), ("交错式", axis_inter)]:
    print(name, ''.join('thw'[a] for a in ax.tolist()))
out = sdpa(apply_rot(q, ids[axis_inter].T * rope_freqs(d)), apply_rot(k, ids[axis_inter].T * rope_freqs(d)), v)
```

3.8 VideoRoPE

Wei et al., 2025 · ICML

针对视频分析 M-RoPE 的不足，提出三点改进：

- **低频时间分配（LTA）**：把**最低频**的维度分给时间轴，避免时间维度高频振荡造成的“周期性误判”
- **对角布局（DL）**：每帧的空间坐标以帧中心对齐，并随帧号整体平移，使视觉 token 沿对角线排布，与文本位置平滑衔接
- **可调时间间隔（ATS）**：用缩放因子 δ 调节帧与帧之间的位置间隔

$$(t, x, y) \mapsto \left(\delta t,\ \ \delta t + x - \tfrac{W}{2},\ \ \delta t + y - \tfrac{H}{2}\right), \qquad \mathcal{S}_t = \{\text{最低频维度}\}$$

```
T, Hp, Wp, delta = 4, 2, 2, 2.0                          # 4 帧 × 2×2 块 = 16 个 token
t_ = (pos // (Hp * Wp)).float(); y_ = ((pos // Wp) % Hp).float(); x_ = (pos % Wp).float()
ids = torch.stack([delta * t_,                           # 时间：可调间隔 δ
                   delta * t_ + x_ - Wp / 2,             # 宽：以中心对齐，并随帧平移（对角布局）
                   delta * t_ + y_ - Hp / 2])            # 高
axis = torch.cat([torch.arange(12) % 2 + 1, torch.zeros(4, dtype=torch.long)])   # 前 12 个(高频)给 x/y 交错，
↪ 最后 4 个(最低频)给 t
out = sdpa(apply_rot(q, ids[axis].T * rope_freqs(d)), apply_rot(k, ids[axis].T * rope_freqs(d)), v)
print(ids[:, ::4].tolist())
```

第 4 章 RoPE 长度扩展

用长度 L 训练的模型，推理到 $L' = sL$ 时，会遇到没见过的旋转角度（主要是低频维度转到了训练中从未到达的相位），导致困惑度飙升。本章的方法都是在**不改模型结构**的前提下，通过调整频率、位置下标或注意力温度来扩展上下文。有的方法完全免训练，有的需要少量微调。

统一视角：RoPE 的角度为 $\phi_i(m) = m \cdot \theta_i$。各方法要么改 m（位置插值、分组、截断），要么改 θ_i（调 base、按频率分段缩放），要么改 softmax 温度。

4.1 调大 base（ABF, Adjusted Base Frequency）

Xiong et al., 2023（Effective Long-Context Scaling）；Rozière et al., 2023（Code Llama, base=10^6）· Llama 3（5×10^5）、Qwen2/3（10^6）

最简单也最常用的做法：在长上下文继续训练时把 base 从 10^4 调到 $10^5 \sim 10^7$。所有频率都变低，最长波长变长，远距离位置不再“转一圈回到原点”。

$$\theta_i = \text{base}^{-2i/d}, \qquad \lambda_{\max} = \frac{2\pi}{\theta_{d/2-1}} = 2\pi \cdot \text{base}^{(d-2)/d}$$

```
for base in [1e4, 5e5, 1e6]:
    th = rope_freqs(128, base)                          # 以 128 维头为例
    print(f"base={base:.0e}  最长波长 ≈ {2 * math.pi / th[-1]:,.0f} tokens")
out = sdpa(rope(q, base=1e6), rope(k, base=1e6), v, causal)
```

4.2 位置插值（Position Interpolation, PI）

Chen et al., 2023 · Meta

不外推，而是**内插**：把位置下标统一压缩 s 倍，使最大位置仍落在训练范围内。只需约 1000 步微调就能扩展到 32K。缺点是高频维度也被压缩了，相邻 token 之间的分辨率下降。

$$\phi_i(m) = \frac{m}{s}\,\theta_i, \qquad s = \frac{L'}{L}$$

```
L, Lp = 8, 16                                           # 训练长度 8，目标长度 16
s = Lp / L
pi_pos = pos / s                                        # 位置 0..15 → 0, 0.5, ..., 7.5
```

```
out = sdpa(rope(q, pi_pos), rope(k, pi_pos), v, causal)
print("最大位置:", pi_pos.max().item(), "< 训练长度", L)
```

4.3 NTK-aware 插值

bloc97, 2023（Reddit/社区）

从神经正切核（NTK）的角度看，PI 压缩高频会丢失细节。NTK-aware 改为调大 base，使**最高频维度几乎不变**（相当于外推）、**最低频维度恰好被压缩 s 倍**（相当于插值），中间维度平滑过渡。这种方法免训练即可使用。

$$\text{base}' = \text{base} \cdot s^{\frac{d}{d-2}} \quad \Rightarrow \quad \theta'_0 = \theta_0, \qquad \theta'_{d/2-1} = \theta_{d/2-1}/s$$

```
s = 4.0
base_ntk = 10000 * s ** (d / (d - 2))
th, th_ntk = rope_freqs(d), rope_freqs(d, base_ntk)
print(f"最高频比值 {th_ntk[0] / th[0]:.3f}，最低频比值 {th_ntk[-1] / th[-1]:.3f} (= 1/s) ")
out = sdpa(rope(q, base=base_ntk), rope(k, base=base_ntk), v, causal)
```

4.4 动态 NTK（Dynamic NTK）

emozilla, 2023 · Qwen（第一代）、Hugging Face `rope_scaling={"type":"dynamic"}`

缩放因子不再固定，而是**随当前序列长度动态计算**：长度没超过 L 时完全不变（短文本性能零损失），超过后才逐步调大 base。

$$\text{base}'(L_{\text{cur}}) = \text{base} \cdot \left(\alpha \frac{L_{\text{cur}}}{L} - (\alpha - 1) \right)^{\frac{d}{d-2}}, \qquad L_{\text{cur}} > L$$

```
def dynamic_base(L_cur, L=8, alpha=1.0, base=10000.0):
    if L_cur <= L: return base                          # 短序列：完全不变
    return base * (alpha * L_cur / L - (alpha - 1)) ** (d / (d - 2))
for L_cur in [4, 8, 12, 16]:
    print(L_cur, f"{dynamic_base(L_cur):,.0f}")
b = dynamic_base(N)
out = sdpa(rope(q, base=b), rope(k, base=b), v, causal)
```

4.5 YaRN

Peng et al., 2023 · ICLR 2024 · Qwen2.5/Qwen3 长上下文、DeepSeek-V3 使用

按每个频率在训练长度内**转过的圈数** r_i 分三段处理：

- **圈数多的高频维度**：直接外推，不做改动
- **圈数少的低频维度**：按 PI 方式插值
- **中间维度**：用线性斜坡平滑过渡

此外，插值会让注意力分布变“平”，YaRN 再给 q、k 各乘一个温度修正系数 $\sqrt{1/t}$。微调所需数据和步

数都比 PI 少一个数量级。

$$r_i = \frac{L\,\theta_i}{2\pi}, \quad \gamma_i = \mathrm{clip}\left(\frac{r_i-\alpha}{\beta-\alpha}, 0, 1\right), \quad \theta_i' = (1-\gamma_i)\frac{\theta_i}{s} + \gamma_i\theta_i, \quad \sqrt{1/t} = 0.1\ln s + 1$$

```
L, s, alpha, beta = 4096, 4.0, 1.0, 32.0                    # 训练长度 4K → 16K
th = rope_freqs(d)
r = L * th / (2 * math.pi)                                  # 每个频率在训练长度内转过的圈数
gam = ((r - alpha) / (beta - alpha)).clamp(0, 1)            # 1 = 外推，0 = 插值
th_yarn = (1 - gam) * th / s + gam * th
mscale = 0.1 * math.log(s) + 1                              # 温度修正（q、k 各乘一次）
yarn = lambda t: apply_rot(t, pos.float()[:, None] * th_yarn) * mscale
out = sdpa(yarn(q), yarn(k), v, causal)
print("各频率处理方式:", ''.join('E' if g == 1 else ('I' if g == 0 else '~') for g in gam.tolist()))
```

4.6 LongRoPE / LongRoPE2

Ding et al., 2024（LongRoPE）；Shang et al., 2025（LongRoPE2）· 微软 · Phi-3 128K 使用

不再用手工公式，而是用**进化搜索**为每个频率维度找一个独立的缩放因子 λ_i（整体上低频维度缩放更多）。另外，开头 $\hat{n}$ 个位置不做缩放，保留局部精度。配合渐进式扩展可以达到 2M 上下文。LongRoPE2 改进了搜索目标（以“大海捞针”式的关键 token 困惑度为准），并采用混合上下文窗口训练，以保住短文本性能。

$$\phi_i(m) = \begin{cases} m\,\theta_i & m < \hat{n} \\ m\,\theta_i/\lambda_i & m \geq \hat{n} \end{cases}, \qquad \{\lambda_i\}, \hat{n} = \arg\min\ \mathrm{PPL}_{\mathrm{long}}$$

```
s, n_hat = 4.0, 4
lam = 1 + (s - 1) * torch.linspace(0, 1, d // 2) ** 2       # 示意：原文由进化搜索得到，低频维度缩放更大
th = rope_freqs(d)
ang = torch.where(pos[:, None] < n_hat, pos[:, None] * th, pos[:, None] * th / lam)   # 开头 n_hat 个位置不缩放
out = sdpa(apply_rot(q, ang), apply_rot(k, ang), v, causal)
print("λ 首末:", round(lam[0].item(), 2), round(lam[-1].item(), 2))
```

4.7 Self-Extend

Jin et al., 2024 · ICML · “LLM Maybe LongLM”

完全免训练的两级注意力：

- **邻近窗口 w 内**：使用正常的相对位置，保住局部精度
- **窗口之外**：把位置**整除分组**（$\lfloor p/G \rfloor$），让远处的相对距离落回训练范围，再加一个偏移，使两段在边界处连续

$$\mathrm{rel}(i,j) = \begin{cases} i-j & i-j < w \\ \lfloor i/G \rfloor - \lfloor j/G \rfloor + (w - \lfloor w/G \rfloor) & \text{否则} \end{cases}$$

```
w, G = 4, 3                                          # 邻近窗口、分组大小
s_near = rope(q) @ rope(k).transpose(-2, -1)         # 正常位置
qg = rope(q, pos // G + (w - w // G))                # 分组位置 + 偏移
kg = rope(k, pos // G)
s_far = qg @ kg.transpose(-2, -1)
s = torch.where((I - J) < w, s_near, s_far) / math.sqrt(d)
out = s.masked_fill(~causal, float('-inf')).softmax(-1) @ v
rel = torch.where((I - J) < w, I - J, I // G - J // G + (w - w // G))
print("最大相对距离:", int(rel[causal].max()), "（原本为", N - 1, "）")
```

4.8 ReRoPE / Leaky ReRoPE

苏剑林, 2023（科学空间博客）

把相对位置**截断**在窗口 w：窗口内正常编码，窗口外一律视为距离 w。Leaky 版本在窗口外让距离以斜率 $1/\kappa$ 缓慢增长。需要计算两次注意力分数，然后拼接。

$$\text{ReRoPE: rel} = \min(i-j,\ w) \qquad \text{Leaky: rel} = \begin{cases} i-j & i-j<w \\ w + \frac{i-j-w}{\kappa} & \text{否则} \end{cases}$$

```
w, kappa = 4, 4.0
s_in = rope(q) @ rope(k).transpose(-2, -1)                # 窗口内：正常 RoPE
s_re = rope(q, torch.full((N,), float(w))) @ k.transpose(-2, -1)         # ReRoPE：q 转 w、k 不转 → 相对距离恒
↪ 为 w
s_lk = rope(q, (pos - w) / kappa + w) @ rope(k, pos / kappa).transpose(-2, -1)   # Leaky：w + (i-j-w)/κ
mask_in = (I - J) < w
for name, s_out in [("ReRoPE", s_re), ("Leaky", s_lk)]:
    s = torch.where(mask_in, s_in, s_out) / math.sqrt(d)
    out = s.masked_fill(~causal, float('-inf')).softmax(-1) @ v
    print(name, out.shape)
```

4.9 双块注意力（Dual Chunk Attention, DCA）

An et al., 2024 · ICML（ChunkLlama）· Qwen2.5-1M 使用（配合 YaRN）

把序列切成长度为 c 的块，按三种情况分别确定相对位置，保证所有相对距离都不超过训练长度：

- **块内**（intra）：查询与键都用块内的局部位置 $p \bmod c$
- **跨块**（inter）：查询位置固定为一个大常数 C，键仍用块内位置
- **相邻块**（successive）：对相邻块做修正，使局部连续

整个方法免训练，可扩展到原长度的数倍以上。

$$\text{rel}(i,j) = \begin{cases} (i \bmod c) - (j \bmod c) & \lfloor i/c \rfloor = \lfloor j/c \rfloor \\ C - (j \bmod c) & \text{否则（}C = L-1\text{）} \end{cases}$$

```
c, L = 4, 8                                          # 块大小、训练长度
same = (I // c) == (J // c)
s_intra = rope(q, pos % c) @ rope(k, pos % c).transpose(-2, -1)
```

```
s_inter = rope(q, torch.full((N,), float(L - 1))) @ rope(k, pos % c).transpose(-2, -1)
s = torch.where(same, s_intra, s_inter) / math.sqrt(d)     # （省略了相邻块修正）
out = s.masked_fill(~causal, float('-inf')).softmax(-1) @ v
rel = torch.where(same, I % c - J % c, (L - 1) - J % c)
print("相对距离范围:", int(rel[causal].min()), "~", int(rel[causal].max()), "（训练长度", L, "）")
```

4.10 LogN 注意力缩放

苏剑林, 2021；Qwen（第一代）；Llama 4 的推理期温度缩放属于同一思路

序列越长，softmax 的分母里候选越多，注意力分布越“平”（熵越大）。给打分乘上 $\log_L n$（n 为当前可见长度），使长度超过训练长度时注意力熵大致保持不变。这个方法常与 NTK、YaRN 配合使用。

$$s_{ij} = \max\left(1,\, \log_L n_i\right) \cdot \frac{q_i^\top k_j}{\sqrt{d}}, \qquad n_i = i + 1$$

```
L = 8
n = (pos + 1).float()
logn = torch.clamp(torch.log(n) / math.log(L), min=1.0)    # 训练长度内为 1，超出后缓慢增大
out = sdpa(rope(q) * logn[:, None], rope(k), v, causal)
print([round(x, 3) for x in logn[[0, 7, 8, 15]].tolist()])
```

第 5 章 无位置编码与混合方案

一个反直觉的发现：**因果掩码本身就泄露了位置信息**。第 i 个 token 恰好能看到 $i+1$ 个 token，模型可以借此“数”出自己的位置。于是出现了干脆不加位置编码（NoPE），以及把 NoPE 层与 RoPE 层交错堆叠的混合方案。

5.1 无位置编码（NoPE）

Haviv et al., 2022；Kazemnejad et al., 2023 · NeurIPS

只用因果注意力、完全不加位置编码的 decoder 也能学好语言模型。理论上，第一层只要对所有可见 token 均匀分配注意力，并让开头的 BOS token 的 value 为 1，输出就恰好是 $1/(i+1)$，绝对位置因此被编码进隐藏状态。在部分长度泛化任务上，NoPE 甚至优于 RoPE 和 ALiBi；但单独使用时，长上下文检索能力较弱。

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} \text{（无任何位置项）}, \qquad \text{若 } s_{ij} \equiv 0,\ v_j = \mathbb{1}[j=0] \ \Rightarrow\ o_i = \frac{1}{i+1}$$

```
v_bos = torch.zeros(1, 1, N, 1); v_bos[..., 0, 0] = 1      # 只有 BOS 的 value 为 1
zero = torch.zeros(1, 1, N, 1)                             # q=k=0 → 打分全 0 → 因果范围内均匀注意力
o = sdpa(zero, zero, v_bos, causal).flatten()
print("隐藏状态恢复出位置:", torch.allclose(o, 1 / (pos + 1.0)))
out = sdpa(q, k, v, causal)                                # NoPE 本身：标准因果注意力，不加任何位置编码
```

5.2 NoPE / RoPE 交错（iRoPE、RNoPE）

Meta, 2025（Llama 4 iRoPE）；Yang et al., 2025（Cohere “Rope to Nope and Back Again”）· Cohere Command A

两种层分工合作：

- **RoPE 层**：配合**局部**注意力（分块或滑窗），负责近距离的语序与局部结构
- **NoPE 层**：做**全局**注意力，不受位置编码外推问题的困扰，负责远距离检索

典型比例为 3:1。Llama 4 还在 NoPE 层上叠加了**推理期温度缩放**：越靠后的位置，打分放大得越多，防止长序列下注意力被摊平。

$$\text{Layer}_\ell = \begin{cases} \text{NoPE} + \text{Global} & \ell \bmod 4 = 3 \\ \text{RoPE} + \text{Local}(w) & \text{否则} \end{cases}, \qquad \tau_i = 1 + \beta \log\left(\left\lfloor \frac{i+1}{\alpha} \right\rfloor + 1\right)$$

```
w, beta, alpha = 4, 0.1, 4                                  # Llama 4 中 α=8192, β=0.1
local = causal & ((I - J) < w)
tau = 1 + beta * torch.log(torch.floor((pos + 1.0) / alpha) + 1)   # 位置越靠后放大越多
h = x
for layer in range(8):
    qh = kh = vh = h.view(B, N, H, d).transpose(1, 2)       # 简化：省略投影与 FFN
    if layer % 4 == 3:                                      # NoPE 全局层
        y = sdpa(qh * tau[:, None], kh, vh, causal)
    else:                                                   # RoPE 局部层
        y = sdpa(rope(qh), rope(kh), vh, local)
    h = h + y.transpose(1, 2).reshape(B, N, D)
out = h; print(out.shape, [round(t, 3) for t in tau[[0, 7, 15]].tolist()])
```

5.3 局部层与全局层使用不同 base

Gemma Team, 2025 · Gemma 3（滑窗层 : 全局层 = 5:1）

Gemma 3 的做法：

- **局部滑窗层**：保持 RoPE base 为 10^4，因为窗口很短，用高分辨率即可
- **全局层**：base 调到 10^6，负责长距离

这样只需要为少数全局层解决长上下文问题，大部分层的 KV cache 也被窗口限住。

$$\text{base}_\ell = \begin{cases} 10^6 & \text{全局层} \\ 10^4 & \text{局部滑窗层} \end{cases}$$

```
w = 4
cfg = {'local': (causal & ((I - J) < w), 1e4), 'global': (causal, 1e6)}
pattern = ['local'] * 5 + ['global']                        # 5:1
h = q
for kind in pattern:
    mask, base = cfg[kind]
    h = h + sdpa(rope(h, base=base), rope(h, base=base), h, mask)
out = h; print(out.shape)
```

第 6 章 数据依赖的位置编码

前面所有方法的位置都只由下标决定，与内容无关。但人理解文本时常按“第几句话”“第几个名词”来定位，而不是“第几个 token”。本章的方法让位置本身**依赖内容**：位置变成可以学习、可以选择性累加的量。

6.1 上下文位置编码（Contextual Position Encoding, CoPE）

Golovneva et al., 2024 · Meta

由内容决定“数什么”：门控 $g_{ij}=\sigma(q_i^\top k_j)$ 决定第 j 个 token 是否“算一个位置”，位置 p_{ij} 是从 j 到 i 的门控值之和，因此可以是小数。小数位置的嵌入由相邻两个整数位置的嵌入插值得到。这样模型可以按词、句子甚至特定标记来计数，在计数、选择性复制等任务上明显优于 RoPE。

$$g_{ij}=\sigma(q_i^\top k_j),\qquad p_{ij}=\sum_{t=j}^{i} g_{it},\qquad s_{ij}=q_i^\top\left(k_j+e[p_{ij}]\right)$$

$$e[p_{ij}]=(p_{ij}-\lfloor p_{ij}\rfloor)\,e[\lceil p_{ij}\rceil]+(\lceil p_{ij}\rceil-p_{ij})\,e[\lfloor p_{ij}\rfloor]$$

```
z = q @ k.transpose(-2, -1)
g = torch.sigmoid(z).masked_fill(~causal, 0)              # 门：这个 token "算不算一个"
p = g.flip(-1).cumsum(-1).flip(-1).clamp(max=N - 1)       # p_ij = Σ_{t=j..i} g_it（小数位置）
E = torch.randn(N, d) * 0.1                               # 整数位置嵌入（可学习）
qe = q @ E.T                                              # 先算 q·e[整数]，再插值，避免逐对计算
pf, pc = p.floor().long(), p.ceil().long()
frac = p - pf
pos_logit = qe.gather(-1, pc) * frac + qe.gather(-1, pf) * (1 - frac)
out = sdpa(q, k, v, causal, bias=pos_logit / math.sqrt(d))
print("最后一个查询看到的小数位置:", [round(t, 2) for t in p[0, 0, -1, -5:].tolist()])
```

6.2 PaTH 注意力

Yang et al., 2025 · “Position Encoding via Accumulating Householder Transformations”

RoPE 的旋转矩阵 R_{n-m} 固定且与内容无关。PaTH 把它换成沿路径的**数据依赖** Householder 变换的累积乘积：q_i 和 k_j 之间经过的每个 token，都会根据自身内容贡献一个反射变换 H_t。这样位置编码本身就能做状态追踪，表达能力严格强于 RoPE，同时仍然有高效的分块并行算法。

$$s_{ij} = q_i^\top \left(\prod_{t=j+1}^{i} H_t \right) k_j, \qquad H_t = I - \beta_t\, w_t w_t^\top, \quad w_t = \frac{W_w x_t}{\|W_w x_t\|},\ \beta_t \in (0,2)$$

```
w_ = F.normalize(torch.randn(B, H, N, d), dim=-1)          # 由 x_t 产生
beta = 2 * torch.sigmoid(torch.randn(B, H, N))
Hs = torch.eye(d) - beta[..., None, None] * w_[..., :, None] * w_[..., None, :]   # (B,H,N,d,d)
s = torch.full((B, H, N, N), float('-inf'))
for i in range(N):                                          # 朴素 O(N²d²)，原文为分块并行算法
    P = torch.eye(d).expand(B, H, d, d)
    for j in range(i, -1, -1):
        s[:, :, i, j] = (q[:, :, i, None] @ P @ k[:, :, j, :, None]).flatten(2).squeeze(-1) / math.sqrt(d)
        P = P @ Hs[:, :, j]                                 # 累积 H_i ... H_j
out = s.softmax(-1) @ v
Hr = torch.eye(d) - 2 * w_[0, 0, 0, :, None] * w_[0, 0, 0, None, :]   # β=2 时是严格的反射
print(out.shape, "| β=2 时 H_t 为正交阵:", torch.allclose(Hr @ Hr.T, torch.eye(d), atol=1e-5))
```

6.3 遗忘门作为位置（Forgetting Transformer, FoX）

Lin et al., 2025 · ICLR

在打分上加数据依赖遗忘门的累积对数 $\sum \log f$，越早的 token 被“遗忘”得越多，衰减速度由内容决定。可以把它看作**数据依赖的 ALiBi**：ALiBi 的固定斜率 m_h 被替换成每步由内容决定的 $-\log f_t$。FoX 不需要任何其它位置编码，长度外推好于 RoPE。

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} + \sum_{l=j+1}^{i} \log f_l, \qquad f_t = \sigma(w_f^\top x_t + b_f) \qquad (\text{ALiBi:}\ \ \log f_l \equiv -m_h)$$

```
logf = F.logsigmoid(torch.randn(B, H, N) + 2)              # 每头每步遗忘门（由 x 产生）
cs = logf.cumsum(-1)
Dmat = cs[..., :, None] - cs[..., None, :]                 # Σ_{l=j+1..i} log f_l
out = sdpa(q, k, v, causal, bias=Dmat)
print("与 ALiBi 的关系：若 log f ≡ -m，则 D_ij = -m(i-j)")
```

6.4 Stick-Breaking 注意力

Tan et al., 2025 · ICLR · IBM

用“折棍过程”取代 softmax：从最近的 token 往回，每个 token 拿走剩余注意力份额的 β_{ij}。由于越远的 token 只能分到被前面拿剩下的部分，**近因偏好是机制本身带来的**，不需要任何位置编码，长度泛化也好。

$$\beta_{ij} = \sigma\left(\frac{q_i^\top k_j}{\sqrt{d}} \right), \qquad A_{ij} = \beta_{ij} \prod_{j<t\le i} (1 - \beta_{it})$$

```
zz = q @ k.transpose(-2, -1) / math.sqrt(d)
logb, log1mb = F.logsigmoid(zz), F.logsigmoid(-zz).masked_fill(~causal, 0)
rev = log1mb.flip(-1).cumsum(-1).flip(-1) - log1mb          # Σ_{t>j} log(1-β_it)
A = torch.exp(logb + rev).masked_fill(~causal, 0)
```

```
out = A @ v
print("行和 ≤ 1（剩余份额可以不分配）:", (A.sum(-1) <= 1 + 1e-5).all().item())
```

第 7 章 隐式位置

7.1 递推结构自带位置（RNN / SSM / 线性注意力）

Gu & Dao, 2023（Mamba）；Lieber et al., 2024（Jamba：注意力层不加位置编码）· Qwen3-Next、Kimi Linear 等混合模型

RNN、状态空间模型（SSM）、线性注意力都是按顺序逐步更新状态的。h_t 取决于 $x_1, \dots, x_t$ 的**先后次序**，再加上衰减门 a_t 带来的近因偏好，位置信息天然就编码在递推里。因此 Jamba 等混合架构中的注意力层可以完全不加位置编码，由 Mamba 层提供位置信息。

$$h_t = a_t \odot h_{t-1} + B_t x_t, \qquad y_t = C_t^\top h_t \qquad \text{(打乱输入顺序} \Rightarrow \text{输出不再只是相应打乱)}$$

```
perm = torch.randperm(N)
attn = lambda z: sdpa(z, z, z)                              # 无位置编码、无掩码的注意力
eq_attn = torch.allclose(attn(q)[:, :, perm], attn(q[:, :, perm]), atol=1e-5)
def ssm(z, a=0.8):                                          # 最简单的线性递推
    h, ys = torch.zeros_like(z[:, :, 0]), []
    for t in range(z.size(2)):
        h = a * h + z[:, :, t]
        ys.append(h)
    return torch.stack(ys, 2)
eq_ssm = torch.allclose(ssm(q)[:, :, perm], ssm(q[:, :, perm]), atol=1e-5)
print("注意力只是跟着打乱（不知道顺序）:", eq_attn, "| SSM 对顺序敏感:", not eq_ssm)
out = ssm(q)
```

附录 速查表

类别	方法	注入位置	训短测长	代表模型
绝对	正弦编码	输入端	差	原始 Transformer
绝对	可学习绝对	输入端	不能	BERT、GPT-2/3、OPT
绝对	随机位置编码	输入端	好	研究
相对	Shaw / Transformer-XL	打分（向量）	中	Transformer-XL、XLNet
相对	T5 Bias	打分（标量）	中	T5、Flan-T5
相对	DeBERTa	打分（解耦）	中	DeBERTa
相对	ALiBi	打分（线性）	好	BLOOM、MPT、Baichuan2-13B
相对	KERPLE / Sandwich / FIRE	打分（函数）	好	研究
旋转	RoPE	旋转 q/k	差（需扩展）	Llama、Qwen、Mistral、DeepSeek、Gemma
旋转	xPos	旋转 + 衰减	中	研究
旋转	部分 RoPE	旋转部分维度	同 RoPE	GPT-NeoX、GPT-J、Phi-2
旋转	解耦 RoPE	独立小分支	同 RoPE	DeepSeek-V2/V3、Kimi K2
旋转	2D / M-RoPE / 交错 MRoPE	多轴旋转	同 RoPE	Qwen2-VL、Qwen2.5-VL、Qwen3-VL
扩展	ABF（调大 base）	改 θ	需微调	Code Llama、Llama 3、Qwen
扩展	PI	改 m	需微调	早期长上下文微调
扩展	NTK / Dynamic NTK	改 base	免训练可用	Qwen（第一代）

类别	方法	注入位置	训短测长	代表模型
扩展	YaRN	分频率 + 温度	少量微调	Qwen2.5/3、DeepSeek-V3
扩展	LongRoPE / LongRoPE2	搜索 λ_i	少量微调	Phi-3
扩展	Self-Extend / ReRoPE / DCA	改相对位置	免训练	Qwen2.5-1M（DCA）
扩展	LogN 缩放	温度	配合使用	Qwen（第一代）、Llama 4（类似）
无 / 混合	NoPE	无（因果掩码）	视任务而定	研究
无 / 混合	NoPE+RoPE 交错	分层	好	Llama 4、Cohere Command A
无 / 混合	局部/全局不同 base	分层	好	Gemma 3
数据依赖	CoPE / PaTH	内容决定位置	好	研究
数据依赖	FoX / Stick-breaking	内容决定衰减	好	研究
隐式	RNN / SSM	递推次序	好	Mamba、Jamba、混合线性模型

一句话总结演进脉络

1. **2017–2019**：从绝对位置（正弦编码、可学习表）走向相对位置（Shaw、Transformer-XL）。
2. **2020–2022**：相对位置简化成标量偏置（T5、ALiBi）；RoPE 出现，用旋转统一了绝对与相对。
3. **2023**：RoPE 成为事实标准，研究重心转向长度扩展：PI、NTK、YaRN、调大 base。
4. **2024**：免训练外推（Self-Extend、DCA）、可搜索的缩放（LongRoPE）、多模态 RoPE（M-RoPE）、KV 压缩友好的解耦 RoPE。
5. **2025–2026**：NoPE/RoPE 分层混合（Llama 4、Command A、Gemma 3）、数据依赖的位置（CoPE、PaTH、FoX），以及靠递推自带位置的线性/SSM 混合架构。